Digital VLSI Design

4-bit Array Multiplier
from RTL to GDSII with Open-Source Tools

SkyWater SKY130 (130 nm), `sky130_fd_sc_hd`

Name	Roll number	Contribution(mostly equally in all sections)
R.Krs Kowshikh Reddy	2024102053	Parts A and D, part A and B in report
Gunda Yaswanth Vardhan	2024102012	Parts B and C, part C and D in report

Repository: https://github.com/Kowshikh-10/dvd-array-multiplier-rtl2gds

Contents

1 Overview **1**

2 Part A: RTL Design and Verification **1**
- 2.1 Block diagram . . . 1
- 2.2 Hand-worked example (task 5) . . . 2
- 2.3 Simulation results . . . 3
- 2.4 Waveform . . . 3

3 Part B: Logic Synthesis with Yosys **4**

4 Part C: Static Timing Analysis with OpenSTA **5**
- 4.1 Maximum frequency . . . 5
- 4.2 Cells on the critical path . . . 6

5 Part D: Physical Design with LibreLane **7**
- 5.1 Runs . . . 7
- 5.2 Results . . . 7

6 Answers to the Questions (Section 10)(Completed Jointly) **8**

1 Overview

We designed a 4-bit × 4-bit unsigned array multiplier in Verilog. It registers its inputs `a` and `b`, builds the 16 partial products with AND gates, adds them in three rows of four full adders (12 in total), and registers the 8-bit product `p`, giving 16 flip-flops and a latency of 2 clock cycles. The design was verified with an exhaustive self-checking testbench, synthesized to SKY130 standard cells with Yosys, timed with OpenSTA, and taken through place and route to a GDSII layout with LibreLane.

Tools and versions

Tool	Version
IIC-OSIC-TOOLS Docker image	`hpretl/iic-osic-tools:2026.09`
Icarus Verilog	12.0 (host, Part A); 14.0 (container, gate-level simulation)
Yosys	0.69
OpenSTA	3.1.0
LibreLane	3.1.0.dev3
OpenROAD	26Q3
Magic	8.3.684
KLayout	0.28.16 (layout screenshots)

2 Part A: RTL Design and Verification

2.1 Block diagram

Figures 1 and 2 show the multiplier. Two 4-bit input registers (8 D flip-flops) feed 16 AND gates that form the partial products $pp_i[j] = a[j] \cdot b[i]$; three rows of four full adders (each row a 4-bit ripple-carry adder) add them; and an 8-bit output register (8 D flip-flops) captures the product. All 16 flip-flops are clocked by `clk` and cleared by the active-low asynchronous reset `rst_n`.

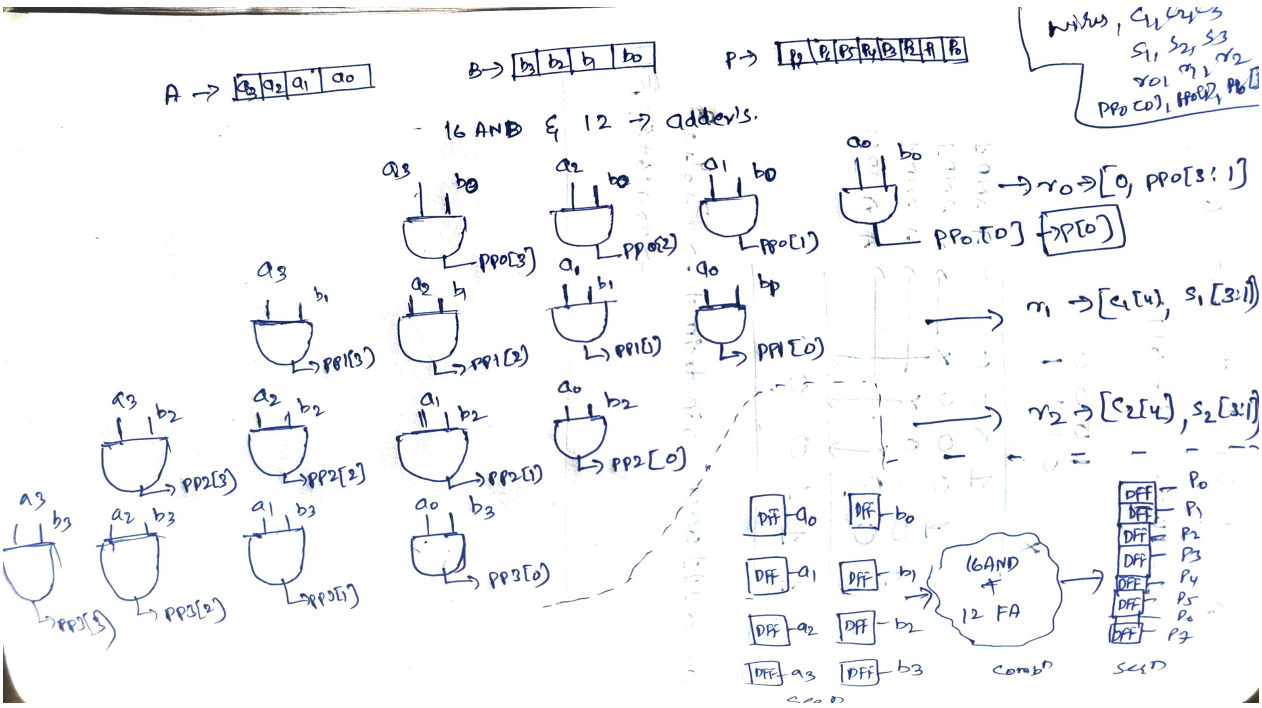


Figure 1: Block diagram (1 of 2): the 16 AND gates producing pp_0–pp_3, the vectors r_0–r_2 passed down between rows, and the input registers (8 DFFs) and output register (8 DFFs) around the combinational logic.

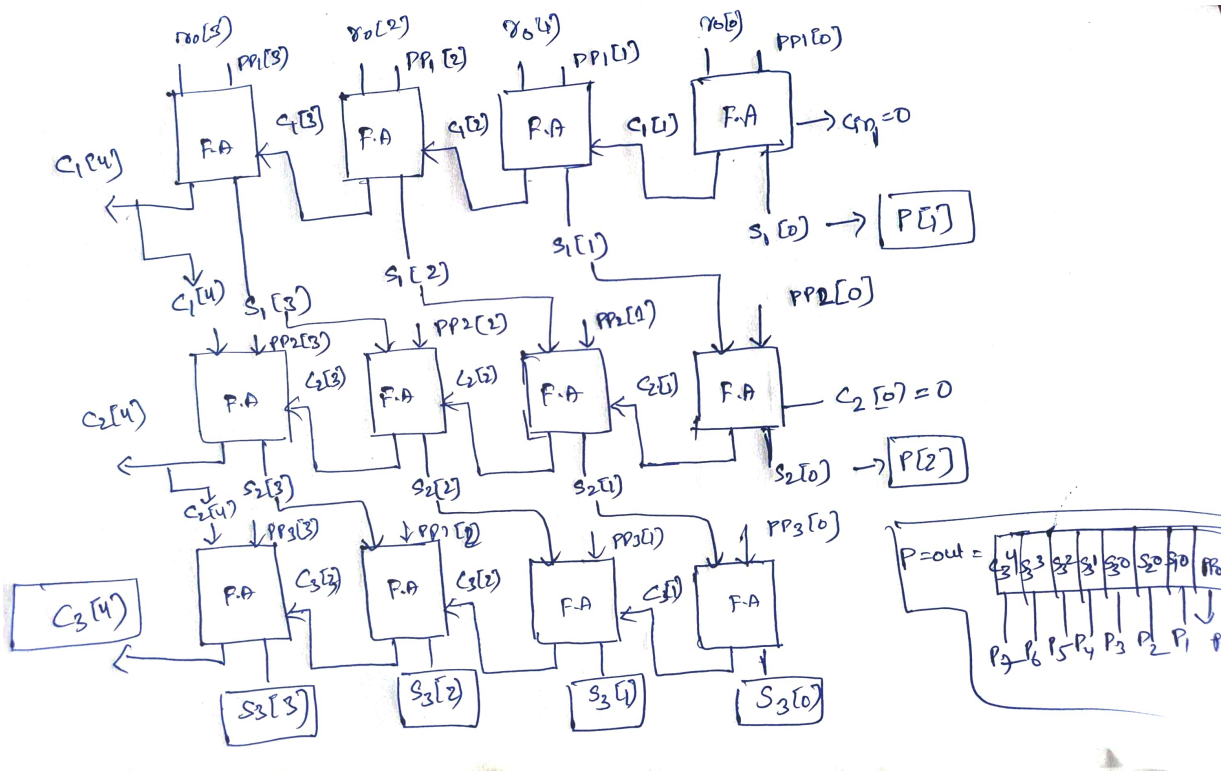


Figure 2: Block diagram (2 of 2): the 12 full adders in three ripple-carry rows, and the product $p = \{c_3[4], s_3[3{:}0], s_2[0], s_1[0], pp_0[0]\}$.

2.2 Hand-worked example (task 5)

Example Kowshikh(2024102053, last digit 3): $A = 3+5 = 8$, and since $A \neq 7$, $B = 7$, so $A \times B = 56$.

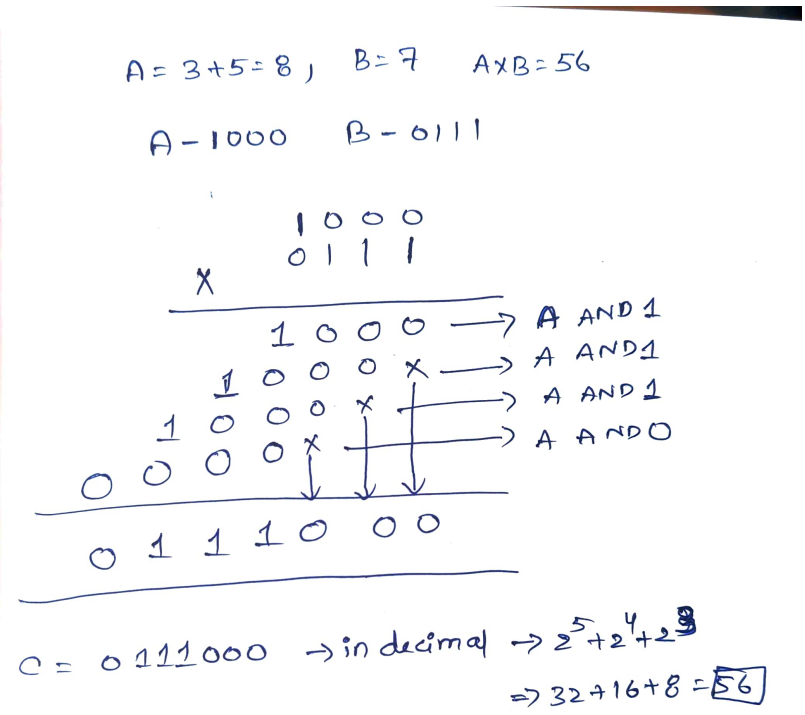


Figure 3: Hand-worked example: $A = 1000_2$ and $B = 0111_2$ give the partial products $A \cdot b_0$, $A \cdot b_1$, $A \cdot b_2$ (each 1000) and $A \cdot b_3 = 0000$, shifted and added to $P = 0011\,1000_2 = 2^5 + 2^4 + 2^3 = 56$.

The same multiplication through the hardware rows (Section 2 of the assignment):

Row	Adds	Sum s	Carry-out	Product bit	Passed down
0	AND gates only: $pp_0 = 1000$	–	–	$p[0] = 0$	$r_0 = 0100$
1	pp_1 1000 + r_0 0100	1100	0	$p[1] = 0$	$r_1 = 0110$
2	pp_2 1000 + r_1 0110	1110	0	$p[2] = 0$	$r_2 = 0111$
3	pp_3 0000 + r_2 0111	0111	0	$p[3] = 1$	$p[7{:}4] = 0011$

So $p = 0011\,1000_2 = 56$. No carry ripples in this example.

2.3 Simulation results

RTL simulation (all 256 input combinations):

```
PASS: 256 vectors checked, 0 errors
```

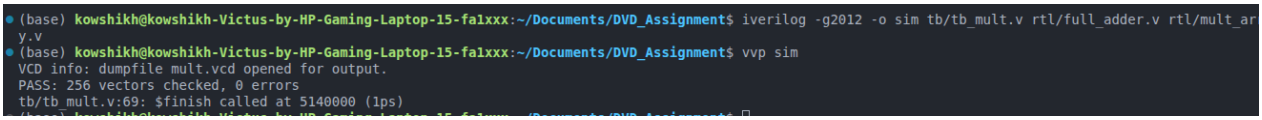


Figure 4: RTL simulation with the exhaustive self-checking testbench.

Error test (task 6): `sum` in `full_adder.v` was temporarily changed to `a ^ b`. The testbench reported:

```
FAIL: 256 vectors checked, 95 errors
```

With `sum = a ^ b`, a full adder only produces a wrong sum when its carry-in is 1. Products in which no carry ever enters an adder are still correct; our own example 8×7 is one of them. Only 95 of the 256 vectors fail, which is why the testbench must check every combination: a few hand-picked tests could miss the bug entirely. The change was undone afterwards and the testbench passed again.

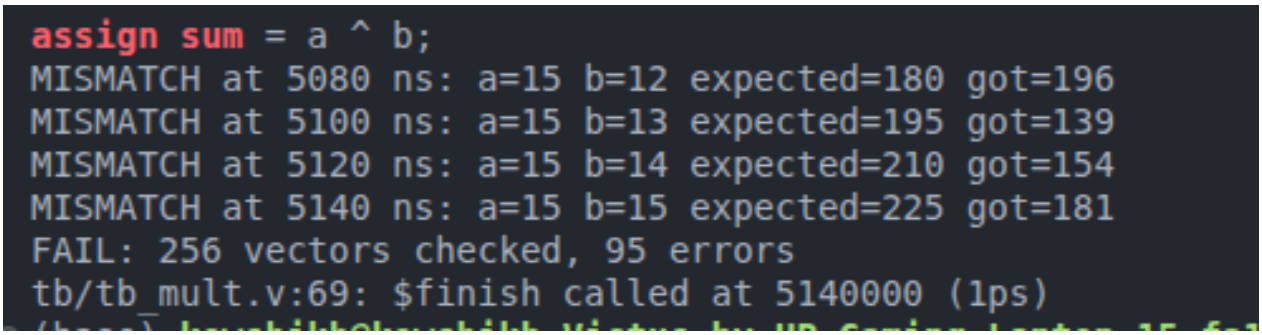


Figure 5: Error test: with `sum = a ^ b` the testbench prints FAIL.

2.4 Waveform

In the waveform, `a` is already 8 and `b` changes from 6 to 7 at 2720 ns. The rising edge at 2725 ns (edge 1) captures the inputs into `a_reg` and `b_reg`, and the rising edge at 2735 ns (edge 2) captures the product, so `p` changes from 48 (the previous test, 8×6) to 56. The latency is therefore 2 clock cycles.

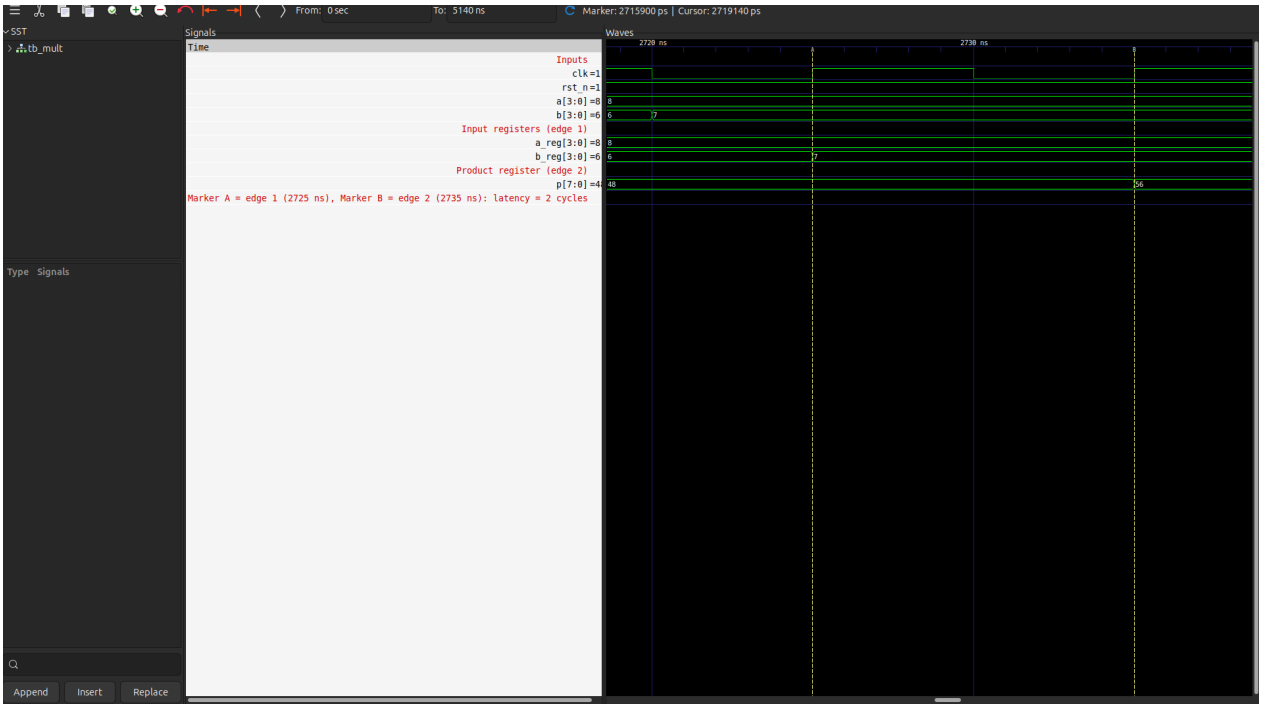


Figure 6: Waveform for 8×7 showing `clk`, `rst_n`, `a`, `b` and `p`, with the 2-cycle latency marked (marker A = edge 1, marker B = edge 2).

3 Part B: Logic Synthesis with Yosys

Table 1: Part B results.

Metric	**Value**
Total number of cells	68
Number of flip-flops	16 (`sky130_fd_sc_hd__dfrtp_1`)
Total cell area (μm^2)	785.754
Flip-flop area (μm^2)	$16 \times 25.024 = 400.384$
Flip-flop area (% of total)	50.96 %
The 3 most-used cell types and their counts	`dfrtp_1` $\times$ 16, `nand2_1` $\times$ 8, `and2_0` and `xnor2_1` $\times$ 6 each (tie)
Netlist simulation result (PASS/FAIL line)	`PASS: 256 vectors checked, 0 errors`

```
7. Printing statistics.

=== mult_array ===

        +----------Local Count, excluding submodules.
        |        +-Local Area, excluding submodules.
        |        |
       59        - wires
       85        - wire bits
       15        - public wires
       41        - public wire bits
        5        - ports
       18        - port bits
       68  785.754 cells
        1    8.758   sky130_fd_sc_hd__a2111oi_0
        3   15.014   sky130_fd_sc_hd__a21oi_1
        3   22.522   sky130_fd_sc_hd__a22oi_1
        6   37.536   sky130_fd_sc_hd__and2_0
        2   17.517   sky130_fd_sc_hd__and4_1
        2    7.507   sky130_fd_sc_hd__clkinv_1
       16  400.384   sky130_fd_sc_hd__dfrtp_1
        2   12.512   sky130_fd_sc_hd__lpflow_isobufsrc_1
        5   50.048   sky130_fd_sc_hd__maj3_1
        2   20.019   sky130_fd_sc_hd__mux2i_1
        8   30.029   sky130_fd_sc_hd__nand2_1
        1    5.005   sky130_fd_sc_hd__nand3_1
        1    6.256   sky130_fd_sc_hd__nand4_1
        4   15.014   sky130_fd_sc_hd__nor2_1
        2    10.01   sky130_fd_sc_hd__o21ai_0
        1    6.256   sky130_fd_sc_hd__or3_1
        6    52.55   sky130_fd_sc_hd__xnor2_1
        2   45.043   sky130_fd_sc_hd__xnor3_1
        1   23.773   sky130_fd_sc_hd__xor3_1

   Chip area for module '\mult_array': 785.753600
     of which used for sequential elements: 400.384000 (50.96%)
```

Figure 7: Cell and area report from `synth/mult_array_stat.rpt`.

```
VCD info: dumpfile mult.vcd opened for output.
PASS: 256 vectors checked, 0 errors
tb/tb_mult.v:69: $finish called at 5140000 (1ps)
```

Figure 8: Gate-level simulation of the synthesized netlist with the SKY130 cell models.

4 Part C: Static Timing Analysis with OpenSTA

4.1 Maximum frequency

With $T = 10$ ns the worst setup slack was WNS $= 7.046$ ns (printed as 7.05 by `report_worst_slack`), so

$$T_{\min} = T - \mathrm{WNS} = 10 - 7.046 = 2.954 \approx 2.95 \text{ ns}, \qquad F_{\max} = \frac{1000}{T_{\min}} = \frac{1000}{2.95} = 338.98 \text{ MHz}.$$

Re-running with $T = 2.95$ ns gave a slack of -0.004 ns, which is effectively 0. It is slightly negative only because 2.954 ns was rounded down to 2.95 ns; with $T = 2.96$ ns the slack is $+0.006$ ns.

Table 2: Part C results.

Metric	Value
Worst slack at $T = 10\,\text{ns}$ (ns)	7.046 (7.05)
$T_{\min}$ (ns)	2.95 (exactly 2.954)
$F_{\max}$ (MHz)	338.98
Slack at $T = T_{\min}$ (ns)	-0.004
Critical path start point	`_106_` = input register `a_reg[2]` (`dfrtp_1`)
Critical path end point	`_103_` = output register `p[7]` (`dfrtp_1`)
Number of cells on the critical path	8 logic cells (10 including the two flip-flops)
Latency (ns) $= 2 \times T_{\min}$	$2 \times 2.95 = 5.90$
Throughput (million results per second)	338.98 (one result per clock cycle)

4.2 Cells on the critical path

Table 3: Critical path, in order (from `sta/mult_array_timing.rpt`, $T = 2.95\,\text{ns}$).

#	Instance	Cell type	Delay (ns)	Arrival (ns)	From a full adder?
start	`_106_` (`a_reg[2]`)	`dfrtp_1` (CLK→Q)	0.399	0.399	–
1	`_052_`	`and4_1`	0.216	0.615	No (AND / row-1 logic)
2	`_054_`	`o21ai_0`	0.180	0.795	No (AND / row-1 logic)
3	`_055_`	`nand3_1`	0.130	0.925	No (AND / row-1 logic)
4	`_066_`	`maj3_1`	0.375	1.299	Yes (carry)
5	`_072_`	`xnor3_1`	0.397	1.696	Yes (sum)
6	`_084_`	`maj3_1`	0.392	2.089	Yes (carry)
7	`_086_`	`maj3_1`	0.385	2.474	Yes (carry)
8	`_087_`	`maj3_1`	0.349	2.823	Yes (carry $c_3[4]$)
end	`_103_` (`p[7]`)	`dfrtp_1` (D)	–	2.823	–

Five of the eight logic cells come from full adders: four `maj3` cells (carry) and one `xnor3` cell (sum). That is fewer than the estimate of about $2N = 8$ full-adder delays. Synthesis implements a full adder's carry as a single `maj3` cell, and it merges the first AND gates and the simplified first-row logic (whose carry-in is the constant 0) into the `and4`, `o21ai` and `nand3` cells. The shape of the path still matches the theory: the signal passes down through the rows and then ripples along the carry chain of the last row to the final carry-out $c_3[4]$, which is product bit `p[7]`.

```
OpenSTA 3.1.0 f89887b596 Copyright (c) 2026, Parallax Software, Inc.
License GPLv3: GNU GPL version 3 <http://gnu.org/licenses/gpl.html>

This is free software, and you are free to change and redistribute it
under certain conditions; type `show_copying' for details.
This program comes with ABSOLUTELY NO WARRANTY; for details type `show_warranty'.
worst slack max 7.05
```

Figure 9: OpenSTA worst setup slack with $T = 10\,\text{ns}$.

```
OpenSTA 3.1.0 f89887b596 Copyright (c) 2026, Parallax Software, Inc.
License GPLv3: GNU GPL version 3 <http://gnu.org/licenses/gpl.html>

This is free software, and you are free to change and redistribute it
under certain conditions; type `show_copying' for details.
This program comes with ABSOLUTELY NO WARRANTY; for details type `show_warranty'.
worst slack max -0.00
```

Figure 10: OpenSTA worst setup slack with $T = T_{\min} = 2.95\,\text{ns}$.

```
Startpoint: _106_ (rising edge-triggered flip-flop clocked by clk)
Endpoint: _103_ (rising edge-triggered flip-flop clocked by clk)
Path Group: clk
Path Type: max

   Delay     Time   Description
---------------------------------------------------------
   0.000    0.000   clock clk (rise edge)
   0.000    0.000   clock network delay (ideal)
   0.000    0.000 ^ _106_/CLK (sky130_fd_sc_hd__dfrtp_1)
   0.399    0.399 v _106_/Q (sky130_fd_sc_hd__dfrtp_1)
   0.216    0.615 v _052_/X (sky130_fd_sc_hd__and4_1)
   0.180    0.795 ^ _054_/Y (sky130_fd_sc_hd__o21ai_0)
   0.130    0.925 v _055_/Y (sky130_fd_sc_hd__nand3_1)
   0.375    1.299 v _066_/X (sky130_fd_sc_hd__maj3_1)
   0.397    1.696 v _072_/X (sky130_fd_sc_hd__xnor3_1)
   0.392    2.089 v _084_/X (sky130_fd_sc_hd__maj3_1)
   0.385    2.474 v _086_/X (sky130_fd_sc_hd__maj3_1)
   0.349    2.823 v _087_/X (sky130_fd_sc_hd__maj3_1)
   0.000    2.823 v _103_/D (sky130_fd_sc_hd__dfrtp_1)
            2.823   data arrival time

   2.950    2.950   clock clk (rise edge)
   0.000    2.950   clock network delay (ideal)
   0.000    2.950   clock reconvergence pessimism
            2.950 ^ _103_/CLK (sky130_fd_sc_hd__dfrtp_1)
  -0.131    2.819   library setup time
            2.819   data required time
---------------------------------------------------------
            2.819   data required time
           -2.823   data arrival time
---------------------------------------------------------
           -0.004   slack (VIOLATED)
```

Figure 11: Critical path report (`sta/mult_array_timing.rpt`).

5 Part D: Physical Design with LibreLane

5.1 Runs

CLOCK_PERIOD started at $1.2 \times T_{\text{min}} = 1.2 \times 2.95 = 3.54$ ns, rounded up to the next 0.5 ns = 4.0 ns, and was increased by 0.5 ns after every run that failed timing.

Table 4: LibreLane attempts.

Run	CLOCK_PERIOD (ns)	Setup slack (ns)	DRC	LVS	Result
1	4.0	−1.706	0	0	Fail (setup)
2	4.5	−1.206	0	0	Fail (setup)
3	5.0	−0.706	0	0	Fail (setup)
4	5.5	−0.206	0	0	Fail (setup)
5	6.0	+0.034	0	0	Pass

DRC and LVS were clean in every run; only setup timing failed. The violations were all in the slow-slow corner `ss_100C_1v60` (slow transistors, 100 °C, 1.60 V). In the typical corner `tt_025C_1v80` that Part C used, the slack was already positive at 4.0 ns (about +1.0 ns). LibreLane signs off at every corner and includes real wire and clock-tree delays, so it needs a longer period than Part C.

5.2 Results

$$F_{\text{max,post-route}} = \frac{1000}{\text{CLOCK_PERIOD} - \text{setup slack}} = \frac{1000}{6.0 - 0.034} = 167.6 \text{ MHz}$$

Table 5: Part D results (from `pnr/metrics.json`).

Metric	Value
CLOCK_PERIOD used (ns)	6.0
Number of runs needed	5
Core area (μm^2)	17 759.5 (die: $150 \times 150 = 22\,500$)
Cells after synthesis (from Part B)	68
Cells after place and route	1 895
Kinds of cells added	1 827 cells: 1 303 decap, 253 fill, 245 tap (`tapvpwrvgnd`), 23 timing-repair buffers (21 `buf`, 2 `clkdlybuf4s50`), 3 clock buffers (`clkbuf`)
Wirelength (μm)	2 398
Setup slack (ns)	+0.034 (worst corner, `ss_100C_1v60`)
Post-route F_{max} (MHz)	167.6
F_{max} from Part C (MHz)	338.98
DRC errors	0 (Magic, KLayout and router)
LVS errors	0

```
/foss/designs/pnr > grep -E '"(design__instance__count|design__core__area|route__wirelength|tim
__lvs_error__count)"' metrics.json
grep '"design__instance__count__class' metrics.json
exit
    "design__instance__count": 1895,
    "timing__setup__ws": 0.03438005533129666,
    "design__core__area": 17759.5,
    "route__drc_errors": 0,
    "route__wirelength": 2398,
    "magic__drc_error__count": 0,
    "klayout__drc_error__count": 0,
    "design__lvs_error__count": 0,
    "design__instance__count__class:inverter": 4,
    "design__instance__count__class:sequential_cell": 16,
    "design__instance__count__class:multi_input_combinational_cell": 48,
    "design__instance__count__class:fill_cell": 1556,
    "design__instance__count__class:tap_cell": 245,
    "design__instance__count__class:timing_repair_buffer": 23,
    "design__instance__count__class:clock_buffer": 3,
```

Figure 12: Values read from `pnr/metrics.json` of the final run.

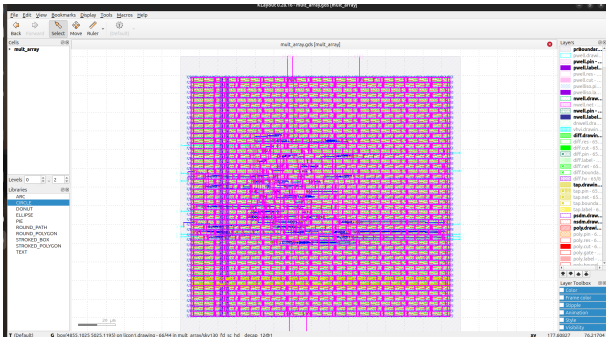


Figure 13: Final layout in KLayout: the whole $150 \times 150\,\mu$m die.

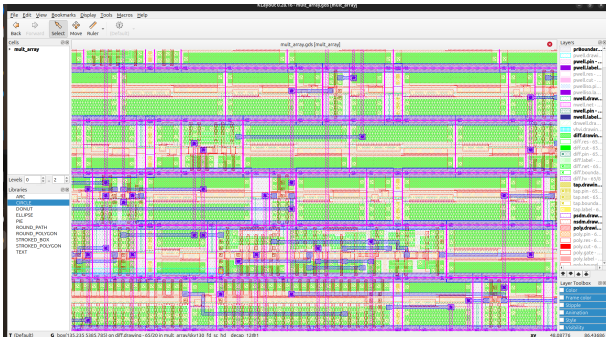


Figure 14: Zoomed view showing standard-cell rows (between the VPWR/VGND rails), diffusion (green), polysilicon (red), metal 1 (blue) and metal 2 (magenta).

6 Answers to the Questions (Section 10)(Completed Jointly)

Q1. *Synthesis produced* $68 - 16 = 52$ *logic cells instead of* $16 + 12 = 28$. *The SKY130 library does contain a full-adder cell (`fa_1`), but it has two outputs, and ABC, the mapper inside Yosys, cannot use multi-output cells (the log warns: "Detected 9 multi-output cells"). Each full adder is therefore built from several single-output gates: `maj3` or AND-OR-INVERT gates for the carry, and `xor/xnor` gates for the sum. ABC also optimizes the flattened design as a whole. It merges AND gates into complex gates (`a22oi`, `and4`, `a2111oi`, ...), simplifies the adders whose inputs are the constant 0 (carry-in* $c[0] = 0$ *and* $r_0[3] = 0$*), and adds a few inverters, so the cells no longer map one-to-one onto the 28 gates of the diagram. The result also depends on how the RTL is written: with the carry in the majority form* $ab + b\,c_{in} + a\,c_{in}$*, which is the same Boolean function, the same flow gives 64 cells,* $751.97\,\mu m^2$ *and* $T_{\min} = 2.60$ *ns instead of 68 cells,* $785.75\,\mu m^2$ *and 2.95 ns, because ABC's optimization is heuristic and depends on the structure it starts from. The 16 flip-flops stay the same.*

Q2. *The critical path starts at the input register `a_reg[2]` (`_106_`) and ends at the output register `p[7]` (`_103_`) through the final carry* $c_3[4]$*. It has 8 logic cells between the two flip-flops (10 cells including them), with a data arrival time of 2.823 ns and a setup time of 0.131 ns, which gives* $T_{\min} = 2.954$ *ns. Five of the eight cells come from full adders (four `maj3` carry cells and*

one `xnor3` *sum cell), fewer than the estimate of* $2N = 8$ *full-adder delays, because one* `maj3` *cell computes a whole carry and the first-row logic is merged with the AND gates. The path follows the expected shape: down through the rows, then along the carry chain of the last row.*

Q3. *The 16 flip-flops take* $400.384/785.754 = 50.96\,\%$ *of the cell area. A single* `dfrtp_1` *flip-flop is 25.02* μm^2*, about 6.7 times the area of a* `nand2_1` *gate (3.75* μm^2*), because it contains two latches, clock inverters and the asynchronous reset. The logic of a 4-bit multiplier is tiny (52 cells, 385.4* μm^2*), so the 16 registers make up half of the area. For a wider multiplier the logic grows with* N^2 *while the registers grow only with* $4N$*, so the flip-flop share would drop.*

Q4. *The post-route* $F_{\max}$ *dropped to 167.6 MHz from the pre-route 338.98 MHz because LibreLane evaluates the design across multiple corners, with the limiting slow-slow corner at 1.60 V and 100°C (*`ss_100C_1v60`*), where gates operate more slowly. In contrast, Part C used only the typical corner (*`tt_025C_1v80`*) with ideal wires and an ideal clock, where the final run still had* $+2.9$ *ns of slack. The routed design also introduces wire resistance and capacitance (2,398* μm *of wire), real clock-tree delays and skew, and buffer delays. To support physical manufacturing and timing, the flow inserted 1,827 additional cells: 1,303 decap cells for supply-voltage stability during switching, 253 fill cells to maintain row continuity and satisfy density rules, 245 tap cells to prevent latch-up, and 26 buffers. Of these, 23 are timing-repair buffers that address transition, load, and timing violations, while 3 are clock buffers that build the clock tree and drive all 16 flip-flops with low skew.?*

Q5. *DRC (design rule check) verifies that the layout geometry obeys the foundry's manufacturing rules: minimum widths and spacings, enclosures and overlaps between layers, density, and antenna rules. LVS (layout versus schematic) extracts the transistors and their connections from the layout and compares them with the synthesized netlist, to prove that the layout is exactly the circuit that was verified. Our final run has 0 DRC errors (Magic, KLayout and the router) and 0 LVS errors. Both must pass because a DRC violation can mean the chip cannot be made reliably (shorts, opens, low yield), while an LVS mismatch means a perfectly manufacturable chip would still be the wrong circuit. In both cases an expensive mask set would be wasted.*

Q6.

The hardware grows roughly with N^2 ($4.67\times$ more full adders), while the critical path grows only linearly with N (about twice as many full-adder delays). From our Part C path, the logic delay is $2.823 - 0.399 = 2.424$ ns for about 8 full-adder delays, roughly 0.30 ns each, plus 0.53 ns of flip-flop overhead (clock-to-Q 0.399 ns and setup 0.131 ns). An 8-bit multiplier would then need about $16 \times 0.30 + 0.53 \approx 5.4$ ns, an $F_{\max}$ of about 185 MHz in the typical corner, compared with 338.98 MHz for 4 bits.

	$N = 4$	$N = 8$	***Ratio***
AND gates, N^2	*16*	*64*	$\times 4$
Full adders, $N(N-1)$	*12*	*56*	$\times 4.67$
Critical path, $\approx 2N$ *FA delays*	*8*	*16*	$\times 2$
Flip-flops, $4N$	*16*	*32*	$\times 2$

The hardware grows roughly with N^2 *(*$4.67\times$ *more full adders), while the critical path grows only linearly with* N *(about twice as many full-adder delays). From our Part C path, the logic delay is* $2.823 - 0.399 = 2.424$ *ns for about 8 full-adder delays, roughly 0.30 ns each, plus 0.53 ns of flip-flop overhead (clock-to-Q 0.399 ns and setup 0.131 ns). An 8-bit multiplier would then need about* $16 \times 0.30 + 0.53 \approx 5.4$ *ns, an* $F_{\max}$ *of about 185 MHz in the typical corner, compared with 338.98 MHz for 4 bits.*